

step01_02_data_quality

October 1, 2026

```
[1]: import pandas as pd
import numpy as np
from pathlib import Path

# =====
# PROJECT PATHS
# =====

BASE_DIR = Path.cwd().parent

DATA_DIR = BASE_DIR / "data_sets"
RESULT_DIR = BASE_DIR / "results"

RESULT_DIR.mkdir(exist_ok=True)

# Pointing to the Indian Railways dataset files
TRAIN_FILE = DATA_DIR / "train_details.csv"
STATION_FILE = DATA_DIR / "station_full_names.csv"
SCHEDULE_FILE = DATA_DIR / "combined_schedule.csv"
DELAY_FILE = DATA_DIR / "combined_delay.csv"

# Number of rows processed at one time for large CSV
CHUNK_SIZE = 500_000
```

```
[2]: # =====
# LOAD SMALL / MEDIUM DATASETS
# =====

train_df = pd.read_csv(TRAIN_FILE)
station_df = pd.read_csv(STATION_FILE)
schedule_df = pd.read_csv(SCHEDULE_FILE)

print("Datasets loaded successfully.")

print("\nTrain Details:")
print(train_df.shape)

print("\nStation Details:")
```

```
print(station_df.shape)

print("\nSchedule:")
print(schedule_df.shape)
```

Datasets loaded successfully.

Train Details:
(8992, 3)

Station Details:
(8963, 4)

Schedule:
(172112, 8)

```
[3]: # =====
# COMPLETE ROW DUPLICATES
# =====

def duplicate_summary(df, dataset_name):

    total_rows = len(df)
    duplicate_rows = df.duplicated().sum()
    duplicate_percentage = (duplicate_rows / total_rows) * 100

    return {
        "Dataset": dataset_name,
        "Total Rows": total_rows,
        "Duplicate Rows": duplicate_rows,
        "Duplicate %": round(duplicate_percentage, 4)
    }

duplicate_results = []

duplicate_results.append(
    duplicate_summary(train_df, "train_details")
)

duplicate_results.append(
    duplicate_summary(station_df, "station_full_names")
)

duplicate_results.append(
    duplicate_summary(schedule_df, "combined_schedule")
)
```

```

duplicate_table = pd.DataFrame(duplicate_results)

print("\n=== COMPLETE ROW DUPLICATE ANALYSIS ===")
print(duplicate_table)

duplicate_table.to_csv(
    RESULT_DIR / "duplicate_summary_small_datasets.csv",
    index=False
)

```

```
=== COMPLETE ROW DUPLICATE ANALYSIS ===
```

	Dataset	Total Rows	Duplicate Rows	Duplicate %
0	train_details	8992	0	0.0
1	station_full_names	8963	0	0.0
2	combined_schedule	172112	0	0.0

```

[4]: # =====
# DELAY DATASET - DUPLICATE ANALYSIS
# =====

delay_total_rows = 0
delay_duplicate_rows = 0

for chunk in pd.read_csv(
    DELAY_FILE,
    chunksize=CHUNK_SIZE
):

    delay_total_rows += len(chunk)

    delay_duplicate_rows += chunk.duplicated().sum()

delay_duplicate_percentage = (
    delay_duplicate_rows / delay_total_rows
) * 100

print("\n=== COMBINED DELAY DUPLICATE ANALYSIS ===")

print("Total rows:", delay_total_rows)
print("Duplicate rows:", delay_duplicate_rows)
print(
    "Duplicate percentage:",
    round(delay_duplicate_percentage, 4),
)

```

=== COMBINED DELAY DUPLICATE ANALYSIS ===

Total rows: 38428703

Duplicate rows: 0

Duplicate percentage: 0.0

```
[5]: # =====  
# MISSING VALUE SUMMARY  
# =====  
  
def missing_value_summary(df, dataset_name):  
  
    result = []  
  
    for column in df.columns:  
  
        total = len(df)  
  
        missing = df[column].isna().sum()  
  
        non_missing = total - missing  
  
        missing_percentage = (  
            missing / total  
        ) * 100  
  
        unique_values = df[column].nunique(  
            dropna=True  
        )  
  
        result.append({  
            "Dataset": dataset_name,  
            "Attribute": column,  
            "Total Rows": total,  
            "Non-Missing": non_missing,  
            "Missing": missing,  
            "Missing %": round(missing_percentage, 4),  
            "Unique Values": unique_values  
        })  
  
    return pd.DataFrame(result)  
  
train_missing = missing_value_summary(  
    train_df,  
    "train_details"  
)
```

```

station_missing = missing_value_summary(
    station_df,
    "station_full_names"
)

schedule_missing = missing_value_summary(
    schedule_df,
    "combined_schedule"
)

small_missing_table = pd.concat(
    [
        train_missing,
        station_missing,
        schedule_missing
    ],
    ignore_index=True
)

print("\n=== MISSING VALUE ANALYSIS ===")
print(small_missing_table.to_string(index=False))

small_missing_table.to_csv(
    RESULT_DIR / "missing_value_summary_small_datasets.csv",
    index=False
)

```

```

=== MISSING VALUE ANALYSIS ===

```

Dataset	Attribute	Total Rows	Non-Missing	Missing
Missing % Unique Values				
train_details	train_no	8992	8992	0
0.0000 8720				
train_details	train_name	8992	8992	0
0.0000 6455				
train_details	type_code	8992	8992	0
0.0000 8				
station_full_names	station_name	8963	8963	0
0.0000 8963				
station_full_names	station_full_name	8963	8963	0
0.0000 8940				
station_full_names	station_zone	8963	8963	0
0.0000 19				
station_full_names	station_address	8963	8869	94
1.0488 6996				
combined_schedule	station_no	172112	172112	0
0.0000 119				
combined_schedule	station_name	172112	172112	0

0.0000	8629				
combined_schedule	distance_from_origin	172112	172112	0	
0.0000	3241				
combined_schedule	arrival_day	172112	172112	0	
0.0000	4				
combined_schedule	arrival_time	172112	163439	8673	
5.0392	1440				
combined_schedule	departure_day	172112	172112	0	
0.0000	4				
combined_schedule	departure_time	172112	163439	8673	
5.0392	1440				
combined_schedule	train_no	172112	172112	0	
0.0000	8673				

```
[6]: # =====
# COMBINED DELAY - MISSING VALUE ANALYSIS
# =====

missing_counts = None
total_rows = 0

for chunk in pd.read_csv(
    DELAY_FILE,
    chunksize=CHUNK_SIZE
):

    total_rows += len(chunk)

    current_missing = chunk.isna().sum()

    if missing_counts is None:
        missing_counts = current_missing
    else:
        missing_counts += current_missing

delay_missing_table = pd.DataFrame({
    "Attribute": missing_counts.index,
    "Total Rows": total_rows,
    "Missing": missing_counts.values
})

delay_missing_table["Non-Missing"] = (
    delay_missing_table["Total Rows"]
    - delay_missing_table["Missing"]
)
```

```

delay_missing_table["Missing %"] = (
    delay_missing_table["Missing"]
    / delay_missing_table["Total Rows"]
    * 100
).round(4)

print("\n=== COMBINED DELAY MISSING VALUE ANALYSIS ===")
print(delay_missing_table.to_string(index=False))

delay_missing_table.to_csv(
    RESULT_DIR / "missing_value_summary_combined_delay.csv",
    index=False
)

```

```

=== COMBINED DELAY MISSING VALUE ANALYSIS ===
  Attribute  Total Rows  Missing  Non-Missing  Missing %
      date    38428703        0    38428703     0.0000
station_no    38428703        0    38428703     0.0000
station_name  38428703        0    38428703     0.0000
      delay    38428703  1874271    36554432     4.8773
    train_no    38428703        0    38428703     0.0000

```

```

[7]: # =====
# MISSING DELAY BY STATION
# =====

station_delay_stats = {}

for chunk in pd.read_csv(
    DELAY_FILE,
    usecols=["station_name", "delay"],
    chunksize=CHUNK_SIZE
):

    grouped = chunk.groupby("station_name")["delay"].agg(
        total_records="size",
        non_missing_delay="count"
    )

    grouped["missing_delay"] = (
        grouped["total_records"]
        - grouped["non_missing_delay"]
    )

    grouped = grouped[
        [

```

```

        "total_records",
        "non_missing_delay",
        "missing_delay"
    ]
]

if len(station_delay_stats) == 0:
    station_delay_stats = grouped
else:
    station_delay_stats = (
        station_delay_stats
        .add(grouped, fill_value=0)
    )

station_delay_stats["missing_percentage"] = (
    station_delay_stats["missing_delay"]
    / station_delay_stats["total_records"]
    * 100
)

station_delay_stats = (
    station_delay_stats
    .sort_values(
        "missing_percentage",
        ascending=False
    )
)

station_delay_stats.to_csv(
    RESULT_DIR / "missing_delay_by_station.csv"
)

print(
    station_delay_stats.head(20)
)

```

station_name	total_records	non_missing_delay	missing_delay \
BNDE	104.0	0.0	104.0
BSCN	52.0	0.0	52.0
HHAL	261.0	0.0	261.0
AJLE	365.0	0.0	365.0
STLA	365.0	0.0	365.0
MLYC	730.0	2.0	728.0
DSBP	365.0	1.0	364.0
NTVT	365.0	1.0	364.0
BKCA	312.0	1.0	311.0

BPHI	365.0	2.0	363.0
JET	313.0	2.0	311.0
MJGP	1460.0	10.0	1450.0
SBID	395.0	3.0	392.0
HSJ	626.0	5.0	621.0
KDV	365.0	3.0	362.0
SYAE	208.0	2.0	206.0
PGDP	417.0	5.0	412.0
KCVL	9281.0	174.0	9107.0
NEM	3911.0	75.0	3836.0
MTN	365.0	7.0	358.0

	missing_percentage
station_name	
BNDE	100.000000
BSCN	100.000000
HHAL	100.000000
AJLE	100.000000
STLA	100.000000
MLYC	99.726027
DSBP	99.726027
NTVT	99.726027
BKCA	99.679487
BPHI	99.452055
JET	99.361022
MJGP	99.315068
SBID	99.240506
HSJ	99.201278
KDV	99.178082
SYAE	99.038462
PGDP	98.800959
KCVL	98.125202
NEM	98.082332
MTN	98.082192

```
[8]: # =====
# MISSING DELAY BY DATE
# =====

date_delay_stats = {}

for chunk in pd.read_csv(
    DELAY_FILE,
    usecols=["date", "delay"],
    chunksize=CHUNK_SIZE
):
```

```

grouped = chunk.groupby("date")["delay"].agg(
    total_records="size",
    non_missing_delay="count"
)

grouped["missing_delay"] = (
    grouped["total_records"]
    - grouped["non_missing_delay"]
)

grouped = grouped[
    [
        "total_records",
        "non_missing_delay",
        "missing_delay"
    ]
]

if len(date_delay_stats) == 0:
    date_delay_stats = grouped
else:
    date_delay_stats = (
        date_delay_stats
        .add(grouped, fill_value=0)
    )

date_delay_stats["missing_percentage"] = (
    date_delay_stats["missing_delay"]
    / date_delay_stats["total_records"]
    * 100
)

date_delay_stats = date_delay_stats.sort_values(
    "missing_percentage",
    ascending=False
)

date_delay_stats.to_csv(
    RESULT_DIR / "missing_delay_by_date.csv"
)

print("\n=== DATES WITH HIGHEST MISSING-DELAY PERCENTAGE ===")
print(date_delay_stats.head(20))

```

```

=== DATES WITH HIGHEST MISSING-DELAY PERCENTAGE ===
      total_records  non_missing_delay  missing_delay \

```

date			
2025-02-25	104672	90445	14227
2025-02-24	105151	91225	13926
2025-02-26	104458	90810	13648
2025-02-18	104688	91639	13049
2025-02-19	104513	91536	12977
2025-02-23	101538	88950	12588
2025-02-27	104810	92061	12749
2025-02-20	104935	92361	12574
2025-02-22	103968	91789	12179
2025-02-16	101587	89843	11744
2025-02-21	105376	93322	12054
2025-02-28	105355	93629	11726
2025-02-17	105136	93544	11592
2025-02-15	104023	92893	11130
2025-02-14	105325	94821	10504
2025-02-13	104704	94394	10310
2025-02-12	104420	94344	10076
2025-02-11	104623	94838	9785
2025-02-08	103820	94278	9542
2025-02-10	105107	95470	9637

	missing_percentage
date	
2025-02-25	13.591983
2025-02-24	13.243811
2025-02-26	13.065538
2025-02-18	12.464657
2025-02-19	12.416637
2025-02-23	12.397329
2025-02-27	12.163916
2025-02-20	11.982656
2025-02-22	11.714181
2025-02-16	11.560534
2025-02-21	11.439037
2025-02-28	11.129989
2025-02-17	11.025719
2025-02-15	10.699557
2025-02-14	9.972941
2025-02-13	9.846806
2025-02-12	9.649492
2025-02-11	9.352628
2025-02-08	9.190907
2025-02-10	9.168752

```
[9]: # =====
# TRAIN NUMBER DUPLICATES
```

```
# =====

duplicate_train_numbers = (
    train_df[
        train_df["train_no"].duplicated(
            keep=False
        )
    ]
    .sort_values("train_no")
)

print("\n=== DUPLICATE TRAIN NUMBERS ===")

print(
    duplicate_train_numbers.head(50)
)

duplicate_train_numbers.to_csv(
    RESULT_DIR / "duplicate_train_numbers.csv",
    index=False
)
```

=== DUPLICATE TRAIN NUMBERS ===

	train_no	train_name	type_code
4549	1042	SNSI DR SPECIAL	EXP-TRAINS
8544	1042	SNSI DR SPECIAL	PRM-TRAINS
8547	1265	MML ABKP SPL	PRM-TRAINS
4556	1265	MML ABKP SPL	EXP-TRAINS
8548	1266	ABKP JBP SPL	PRM-TRAINS
4557	1266	ABKP JBP SPL	EXP-TRAINS
4558	1271	ET BPL SPECIAL	EXP-TRAINS
8549	1271	ET BPL SPECIAL	PRM-TRAINS
8550	1272	BPL ET SPECIAL	PRM-TRAINS
4559	1272	BPL ET SPECIAL	EXP-TRAINS
4560	1435	SUR LTT SPECIAL	EXP-TRAINS
8551	1435	SUR LTT SPECIAL	PRM-TRAINS
8552	1436	LTT SUR SPECIAL	PRM-TRAINS
4561	1436	LTT SUR SPECIAL	EXP-TRAINS
4566	1595	KAWR MAO SPL	EXP-TRAINS
8553	1595	KAWR MAO SPL	PRM-TRAINS
8554	1596	MAO KAWR SPL	PRM-TRAINS
4567	1596	MAO KAWR SPL	EXP-TRAINS
4568	1665	RKMP AGTL SPL	EXP-TRAINS
8555	1665	RKMP AGTL SPL	PRM-TRAINS
8556	1666	AGTL RKMP SPL	PRM-TRAINS
4569	1666	AGTL RKMP SPL	EXP-TRAINS
8228	2003	NDLS SWARNA SHATABDI	SHT-TRAINS

8557	2003	NDLS	SWARNA	SHATABDI	PRM-TRAINS	
8558	2004	LJN	SWARNA	SHATABDI	PRM-TRAINS	
8229	2004	LJN	SWARNA	SHATABDI	SHT-TRAINS	
8559	2005		KALKA	SHTBDI	SPL	PRM-TRAINS
8230	2005		KALKA	SHTBDI	SPL	SHT-TRAINS
8560	2006		KALKA	SHTBDI	SPL	PRM-TRAINS
8231	2006		KALKA	SHTBDI	SPL	SHT-TRAINS
8232	2011	NDLS	KLK	SHATABDI	SHT-TRAINS	
8561	2011	NDLS	KLK	SHATABDI	PRM-TRAINS	
8562	2012		KLK	SHTBDI	SPL	PRM-TRAINS
8233	2012		KLK	SHTBDI	SPL	SHT-TRAINS
8234	2013		ASR	SHTBDI	SPL	SHT-TRAINS
8563	2013		ASR	SHTBDI	SPL	PRM-TRAINS
8564	2014	NDLS		SHATABDI	SPL	PRM-TRAINS
8235	2014	NDLS		SHATABDI	SPL	SHT-TRAINS
8236	2033	CNB	NDLS	SHT	SPL	SHT-TRAINS
8565	2033	CNB	NDLS	SHT	SPL	PRM-TRAINS
8566	2034	NDLS	CNB	SHT	SPL	PRM-TRAINS
8237	2034	NDLS	CNB	SHT	SPL	SHT-TRAINS
8238	2039	NDLS		SHATABDI	SPL	SHT-TRAINS
8567	2039	NDLS		SHATABDI	SPL	PRM-TRAINS
8568	2040	KGM		SHATABDI	SPL	PRM-TRAINS
8239	2040	KGM		SHATABDI	SPL	SHT-TRAINS
8595	2431	TVC	NZM	RAJ	EXP	PRM-TRAINS
8175	2431	TVC	NZM	RAJ	EXP	RAJ-TRAINS
8176	2432	NZM	TVC	RAJ	SPL	RAJ-TRAINS
8596	2432	NZM	TVC	RAJ	SPL	PRM-TRAINS

```
[10]: train_number_summary = (
    train_df
    .groupby("train_no")
    .agg(
        row_count=("train_no", "size"),
        train_names=("train_name", "nunique"),
        type_codes=("type_code", "nunique")
    )
)

print("\n=== TRAIN NUMBER KEY ANALYSIS ===")

print(
    train_number_summary
    .query("row_count > 1")
    .head(50)
)
```

=== TRAIN NUMBER KEY ANALYSIS ===

train_no	row_count	train_names	type_codes
1042	2	1	2
1265	2	1	2
1266	2	1	2
1271	2	1	2
1272	2	1	2
1435	2	1	2
1436	2	1	2
1595	2	1	2
1596	2	1	2
1665	2	1	2
1666	2	1	2
2003	2	1	2
2004	2	1	2
2005	2	1	2
2006	2	1	2
2011	2	1	2
2012	2	1	2
2013	2	1	2
2014	2	1	2
2033	2	1	2
2034	2	1	2
2039	2	1	2
2040	2	1	2
2431	2	1	2
2432	2	1	2
2454	2	1	2
3182	2	1	2
4849	2	1	2
4850	2	1	2
4851	2	1	2
4852	2	1	2
4879	2	1	2
5013	2	1	2
5121	2	1	2
5122	2	1	2
5311	2	1	2
5312	2	1	2
5483	2	1	2
5509	2	1	2
5510	2	1	2
5909	2	1	2
6269	2	1	2
6270	2	1	2
6503	2	1	2
6525	2	1	2
6526	2	1	2

7029	2	1	2
7030	2	1	2
7042	2	1	2
7605	2	1	2

```
[11]: # =====
# STATION KEY ANALYSIS
# =====

station_key_summary = {
    "Total rows": len(station_df),
    "Unique station_name": station_df["station_name"].nunique(),
    "Duplicate station_name rows": (
        station_df["station_name"].duplicated().sum()
    )
}

print("\n=== STATION KEY ANALYSIS ===")

for key, value in station_key_summary.items():
    print(f"{key}: {value}")
```

=== STATION KEY ANALYSIS ===

Total rows: 8963

Unique station_name: 8963

Duplicate station_name rows: 0

```
[12]: train_master_keys = set(
    train_df["train_no"]
    .astype(str)
)

schedule_train_keys = set(
    schedule_df["train_no"]
    .astype(str)
)

schedule_missing_train_keys = (
    schedule_train_keys
    - train_master_keys
)

train_master_unused_keys = (
    train_master_keys
    - schedule_train_keys
)
```

```

print("\n=== TRAIN REFERENTIAL INTEGRITY ===")

print(
    "Train numbers in train_details:",
    len(train_master_keys)
)

print(
    "Train numbers in schedule:",
    len(schedule_train_keys)
)

print(
    "Schedule train numbers NOT in train_details:",
    len(schedule_missing_train_keys)
)

print(
    "Train numbers in train_details NOT in schedule:",
    len(train_master_unused_keys)
)

```

```

=== TRAIN REFERENTIAL INTEGRITY ===
Train numbers in train_details: 8720
Train numbers in schedule: 8673
Schedule train numbers NOT in train_details: 0
Train numbers in train_details NOT in schedule: 47

```

```

[13]: # =====
# UNIQUE TRAIN NUMBERS IN DELAY DATA
# =====

delay_train_keys = set()

for chunk in pd.read_csv(
    DELAY_FILE,
    usecols=["train_no"],
    chunksize=CHUNK_SIZE
):

    delay_train_keys.update(
        chunk["train_no"]
        .astype(str)
        .unique()
    )

```

```
print("\nUnique train numbers in delay:",
      len(delay_train_keys))
```

Unique train numbers in delay: 7033

```
[14]: delay_missing_train_keys = (
        delay_train_keys
        - train_master_keys
    )

    train_not_in_delay = (
        train_master_keys
        - delay_train_keys
    )

    print(
        "Delay train numbers NOT in train_details:",
        len(delay_missing_train_keys)
    )

    print(
        "Train numbers in train_details NOT in delay:",
        len(train_not_in_delay)
    )
```

Delay train numbers NOT in train_details: 0

Train numbers in train_details NOT in delay: 1687

```
[15]: # =====
# STATION REFERENTIAL INTEGRITY - SCHEDULE
# =====

station_master_keys = set(
    station_df["station_name"]
    .astype(str)
)

schedule_station_keys = set(
    schedule_df["station_name"]
    .astype(str)
)

schedule_unknown_stations = (
    schedule_station_keys
    - station_master_keys
)
```

```

unused_station_master_keys = (
    station_master_keys
    - schedule_station_keys
)

print("\n=== STATION REFERENTIAL INTEGRITY ===")

print(
    "Stations in station master:",
    len(station_master_keys)
)

print(
    "Stations in schedule:",
    len(schedule_station_keys)
)

print(
    "Schedule stations NOT in station master:",
    len(schedule_unknown_stations)
)

print(
    "Master stations NOT appearing in schedule:",
    len(unused_station_master_keys)
)

```

```

=== STATION REFERENTIAL INTEGRITY ===
Stations in station master: 8963
Stations in schedule: 8629
Schedule stations NOT in station master: 1
Master stations NOT appearing in schedule: 335

```

```

[16]: # =====
# UNIQUE STATIONS IN DELAY DATA
# =====

delay_station_keys = set()

for chunk in pd.read_csv(
    DELAY_FILE,
    usecols=["station_name"],
    chunksize=CHUNK_SIZE
):

    delay_station_keys.update(

```

```

        chunk["station_name"]
        .astype(str)
        .unique()
    )

print(
    "\nUnique stations in delay:",
    len(delay_station_keys)
)

delay_unknown_stations = (
    delay_station_keys
    - station_master_keys
)

master_stations_without_delay = (
    station_master_keys
    - delay_station_keys
)

print(
    "Delay stations NOT in station master:",
    len(delay_unknown_stations)
)

print(
    "Master stations NOT appearing in delay:",
    len(master_stations_without_delay)
)

```

Unique stations in delay: 8227
 Delay stations NOT in station master: 63
 Master stations NOT appearing in delay: 799

```

[17]: # =====
# NATURAL KEY DUPLICATE CHECK
# =====

natural_key = [
    "date",
    "train_no",
    "station_no"
]

duplicate_key_count = 0

```

```

for chunk in pd.read_csv(
    DELAY_FILE,
    usecols=natural_key,
    chunksize=CHUNK_SIZE
):

    duplicate_key_count += (
        chunk.duplicated(
            subset=natural_key
        ).sum()
    )

print(
    "\nDuplicate natural keys within chunks:",
    duplicate_key_count
)

```

Duplicate natural keys within chunks: 0

```

[18]: # =====
# RELATIONSHIP SUMMARY
# =====

relationship_summary = pd.DataFrame([
    {
        "Relationship":
            "train_details → combined_schedule",
        "Parent Key":
            "train_no",
        "Child Key":
            "train_no",
        "Unmatched Child Keys":
            len(schedule_missing_train_keys),
        "Status":
            (
                "PASS"
                if len(schedule_missing_train_keys) == 0
                else "CHECK"
            )
    },
    {
        "Relationship":
            "train_details → combined_delay",
        "Parent Key":
            "train_no",
        "Child Key":

```

```

        "train_no",
        "Unmatched Child Keys":
            len(delay_missing_train_keys),
        "Status":
            (
                "PASS"
                if len(delay_missing_train_keys) == 0
                else "CHECK"
            )
    },
    {
        "Relationship":
            "station_full_names → combined_schedule",
        "Parent Key":
            "station_name",
        "Child Key":
            "station_name",
        "Unmatched Child Keys":
            len(schedule_unknown_stations),
        "Status":
            (
                "PASS"
                if len(schedule_unknown_stations) == 0
                else "CHECK"
            )
    },
    {
        "Relationship":
            "station_full_names → combined_delay",
        "Parent Key":
            "station_name",
        "Child Key":
            "station_name",
        "Unmatched Child Keys":
            len(delay_unknown_stations),
        "Status":
            (
                "PASS"
                if len(delay_unknown_stations) == 0
                else "CHECK"
            )
    }
])

print("\n=== RELATIONSHIP SUMMARY ===")
print(
    relationship_summary.to_string(index=False)

```

```
)

relationship_summary.to_csv(
    RESULT_DIR / "relationship_summary.csv",
    index=False
)
```

```
=== RELATIONSHIP SUMMARY ===
```

	Relationship	Parent Key	Child Key	Unmatched
Child Keys Status				
	train_details → combined_schedule	train_no	train_no	
0 PASS				
	train_details → combined_delay	train_no	train_no	
0 PASS				
	station_full_names → combined_schedule	station_name	station_name	
1 CHECK				
	station_full_names → combined_delay	station_name	station_name	
63 CHECK				

```
[19]: # =====
# STATION_NO INTERPRETATION
# =====

station_no_train_counts = (
    schedule_df
    .groupby("station_no")["train_no"]
    .nunique()
)

print(
    station_no_train_counts.head(20)
)
```

```
station_no
1      8673
2      8673
3      8542
4      8417
5      8285
6      8082
7      7853
8      7556
9      7242
10     6890
11     6543
12     6187
13     5859
```

```
14    5530
15    5154
16    4849
17    4534
18    4254
19    3890
20    3598
Name: train_no, dtype: int64
```

```
[20]: print(
        "\nNumber of unique station_name values:",
        schedule_df["station_name"].nunique()
    )

    print(
        "Number of unique station_no values:",
        schedule_df["station_no"].nunique()
    )
```

```
Number of unique station_name values: 8629
Number of unique station_no values: 119
```

```
[ ]:
```